

PART5. 컨테이너 애플리케이션 구성

클라우드 네이티브 애플리케이션

클라우드 (Cloud)



클라우드

- 다른 회사의 서버를 빌려서 운영
- 다른 회사가 모두에게 서버를 빌려줄 경우 : 퍼블릭 클라우드(Public Cloud)
- 다른 회사가 특정 조직에게만 서버를 빌려줄 경우: 프라이빗 클라우드(Private Cloud)
- 사용 요청 즉시 서버를 생성(Provisioning)
- 실제 사용한 시간 만큼만 비용 지불

클라우드: 현대 애플리케이션이 겪는 다양한 문제들을 클라우드 환경 구성을 통해 해결

1. 트래픽이 증가할 때 빠르게 대처할 수 있는가? (확장성, Scalability)

클라우드 환경에서는 서버 추가가 10분 내로 이루어집니다. 온프레미스에서는 서버가 미리 준비되어 있지 않은 경우 새로운 서버를 증가하는데 오랜 시간이 소요됩니다.(주문, 배송, 설치 등)

2. 장애 발생 시 빠르게 복구할 수 있는가? (복원력, Resilience)

클라우드 환경에서는 백업 및 복구가 빠르게 이루어질 수 있습니다. (Disaster Recovery)
장애에 대응하기 위한 다양한 지역의 서버를 구축할 수 있습니다.

3. 운영 비용을 효율적으로 운영할 수 있는가?

사용한 만큼만 지불할 수 있기 때문에 운영 비용에 더 효율적입니다.

- 클라우드: 복잡한 대형 애플리케이션이 겪는 다양한 문제들을 클라우드 환경 구성을 통해 해결
- 클라우드 네이티브 애플리케이션: 클라우드 환경을 더 잘 활용할 수 있는 애플리케이션 구조

1. MSA

트래픽 증가에 빠르게 대처하기 위해선 애플리케이션이 MSA 구조로 개발되어야 합니다.

2. 컨테이너 (Container)

컨테이너를 활용해 실행 환경에 종속되지 않는 동작이 보장되어야 합니다.

3. 상태비저장 (Stateless)

애플리케이션 서버는 상태를 가지지 않아야 합니다.

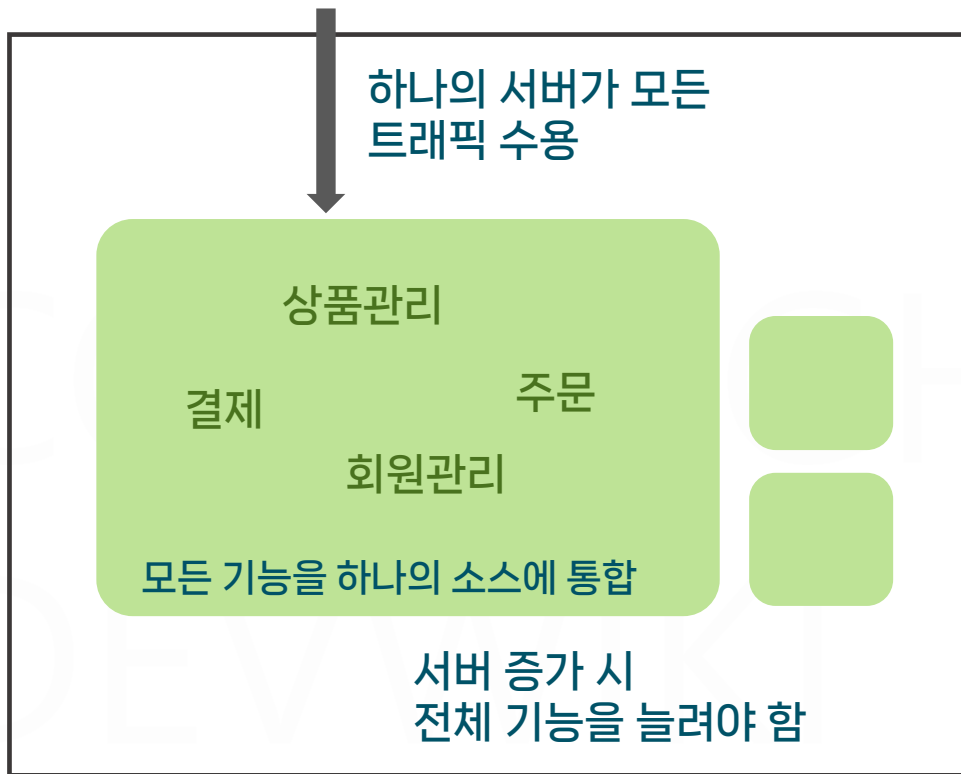
상태를 가지지 않는 애플리케이션은 어디에나 즉시 배포될 수 있습니다.

4. DevOps 및 CI/CD

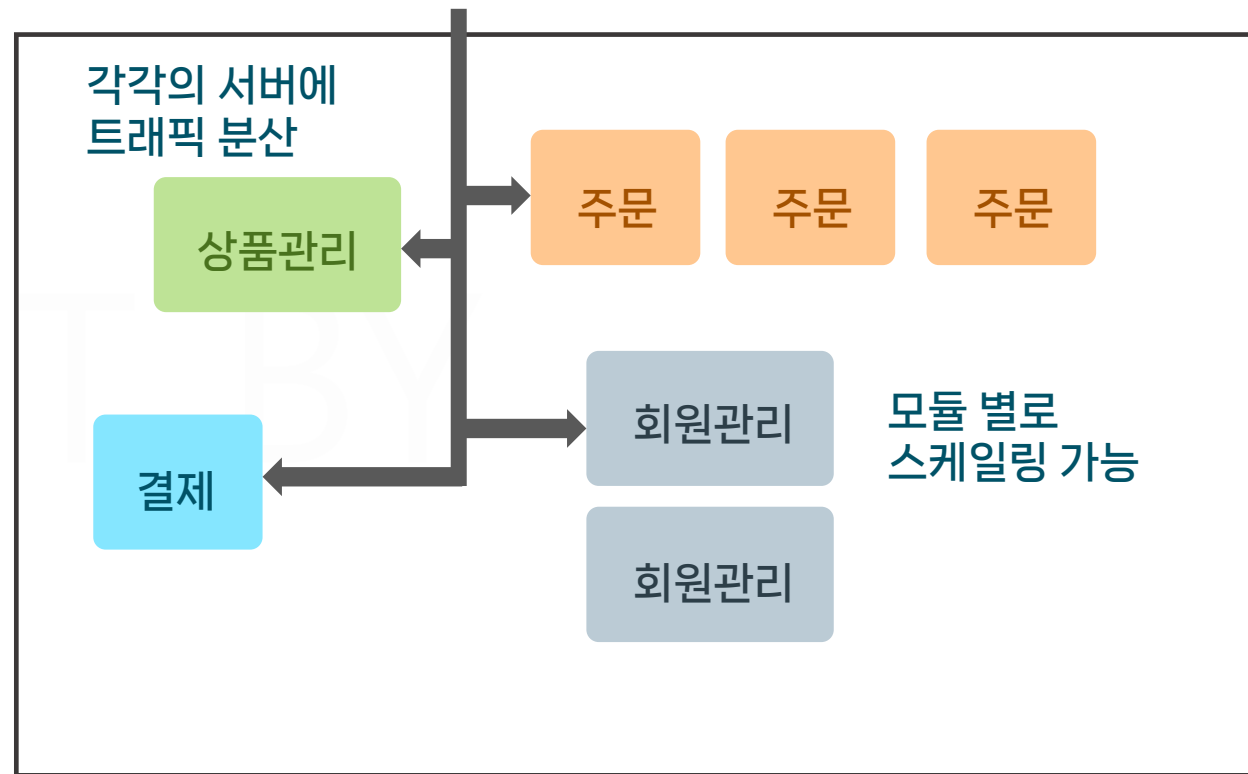
배포가 자동화되어야 하고 빠르게 릴리즈가 수행되어야 합니다.

참고:
<https://12factor.net/ko/>

- 클라우드: 복잡한 대형 애플리케이션이 겪는 다양한 문제들을 클라우드 환경 구성을 통해 해결
- 클라우드 네이티브 애플리케이션: 클라우드 환경을 더 잘 활용할 수 있는 애플리케이션 구조
- MSA:



모놀리식(Monolithic)



마이크로서비스아키텍처(MSA)

항목	모놀리식 아키텍처	마이크로서비스 아키텍처
구조	단일 코드베이스와 애플리케이션	기능별로 독립된 서비스
배포	전체 재배포 필요	개별 서비스 독립 배포
확장성	수직 확장(Scale-Up)을 주로 사용	수평 확장(Scale-Out) 용이
개발 속도	초반 개발 속도 빠름, 코드베이스 커질수록 느려짐	초기 구성 복잡하고 오래 걸림
장애 영향도	전체 애플리케이션 영향	영향이 적고 분리 가능
기술 스택	일반적으로 하나의 스택	서비스별 다양한 스택 가능
유지보수	전체 코드 이해 필요	각 서비스 별로 개발 및 유지보수
복잡성	낮음	서비스 간 통신, 데이터 일관성 유지 등 복잡성 매우 높음

PART5. 컨테이너 애플리케이션 구성

Leafy 애플리케이션 소개

LEAFY

식물 관리 웹 애플리케이션

식물 정보
제공

나의 식물
리스트

식물
다이어리

물 주기
계산

LEAFY

식물 관리 웹 애플리케이션

Leafy
-front

- Vue.js 3 기반 프론트엔드 소스
- Nginx 웹 서버 배포

Leafy

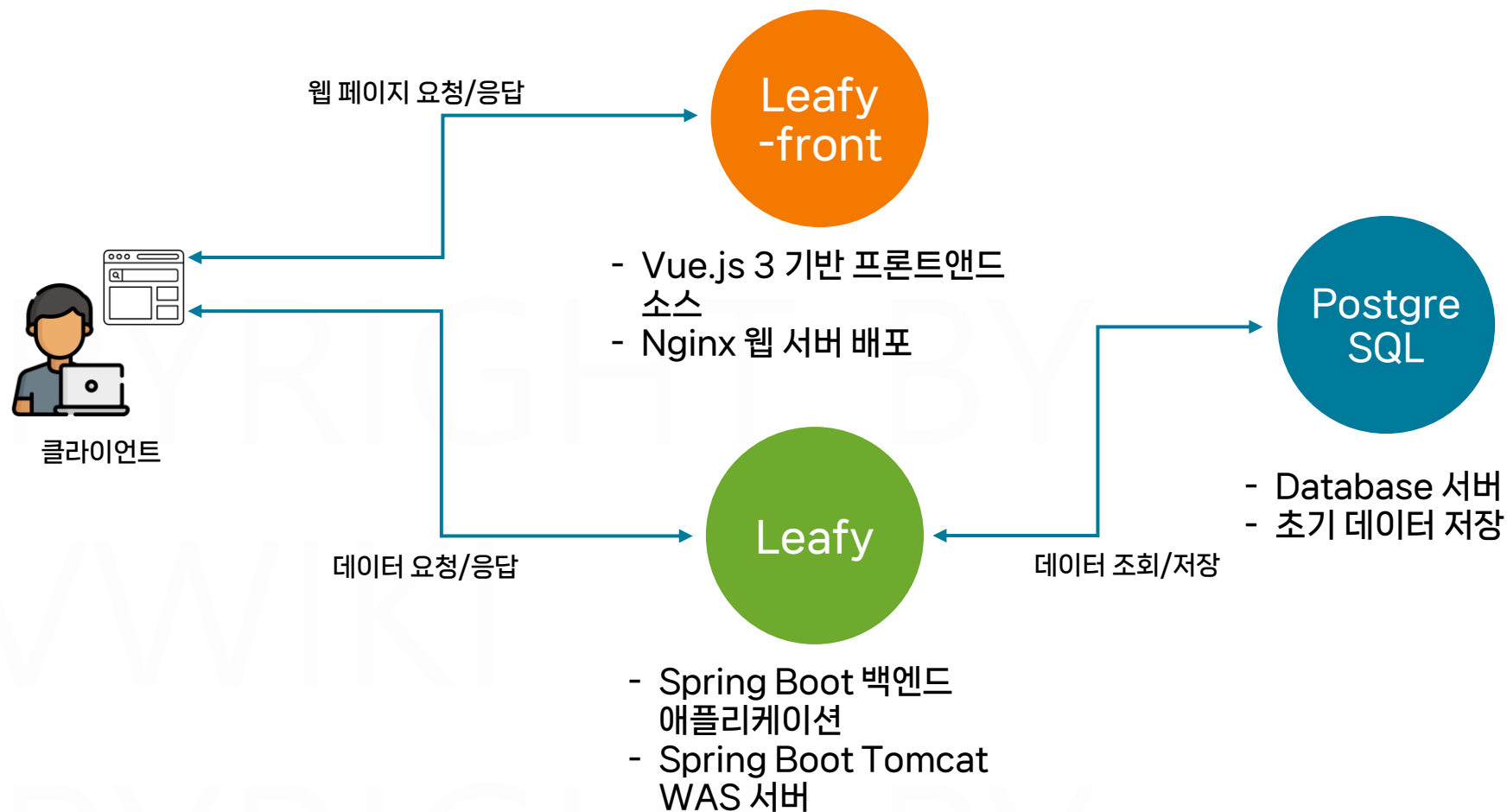
- Spring Boot 백엔드 애플리케이션
- Spring Boot Tomcat WAS 서버

Postgre
SQL

- Database 서버
- 초기 데이터 저장

LEAFY

식물 관리 웹 애플리케이션



Leafy 애플리케이션 구성 및 실행

1. 애플리케이션 구성 및 localhost 접속 테스트

ID: john123@gmail.com / PW: password123

```
docker network create leafy-network
```

```
docker run -d --name leafy-postgres --network leafy-network devwikirepo/leafy-postgres:1.0.0
```

```
docker logs -f leafy-postgres
```

```
docker run -d -p 8080:8080 -e DB_URL=leafy-postgres --network leafy-network --name leafy devwikirepo/leafy-backend:1.0.0
```

```
docker run -d -p 80:80 --network leafy-network --name leafy-front devwikirepo/leafy-frontend:1.0.0
```

```
docker ps
```

2. 구성 환경 삭제

```
docker rm -f leafy-front leafy leafy-postgres
```

PART5. 컨테이너 애플리케이션 구성

Leafy 데이터베이스 컨테이너

LEAFY

식물 관리 웹 애플리케이션

PostgreSQL

사용자 커스터마이징 데이터베이스 구성

4. 데이터베이스 실행
(postgres)

CMD ["postgres", "-c",
"config_file=/etc/postgresql/custom.conf"]

3. SQL문 작성

init.sql

COPY ./init/init.sql
/docker-entrypoint-initdb.d/

2. 환경 설정 파일 작성

/etc/postgresql/custom.conf

COPY ./config/postgresql.conf
/etc/postgresql/custom.conf

1. OS구성 및 PostgreSQL 설치

PostgreSQL

FROM postgres:13

OS

LEAFY

식물 관리 웹 애플리케이션

Postgre
SQL

LEAFY-POSTGRESQL

config

postgresql.conf

init

init.sql

Dockerfile

LICENSE

script.sh

소스코드 다운로드 확인

1. easydocker 폴더에서 소스코드 다운로드

```
git clone https://github.com/daintree-henry/leafy.git
```

2. leafy/leafy-postgresql 디렉토리로 이동, 파일 구조 확인

```
cd leafy/leafy-postgresql
```

```
git switch 00-init (코드 작성을 직접 하고 싶을 경우 실행)
```

```
git switch 01-dockerfile (코드 작성을 건너 뛰고 싶을 경우 실행)
```

leafy/leafy-postgresql/config/postgresql.conf

```
# POSTGRESQL.CONF FILE  
# -----
```

CONNECTIONS AND AUTHENTICATION

```
listen_addresses = '*'          # IP 주소, 호스트명 또는 '*'로 모든 IP에 대한 연결을 허용합니다.  
max_connections = 100           # 동시 접속자 수 제한  
authentication_timeout = 5m     # 인증 시간 초과 시간 (5분)  
password_encryption = md5       # 패스워드 암호화 방식
```

```
...
```

LEAFY

식물 관리 웹 애플리케이션

Postgre
SQL

LEAFY-POSTGRESQL

config

postgresql.conf

init

init.sql

Dockerfile

LICENSE

script.sh

leafy/leafy-postgresql/init/init.sql

각 테이블 생성 및 초기 데이터 입력

```
CREATE TABLE users (  
    user_id SERIAL PRIMARY KEY,  
    ...  
);  
  
-- Plants table creation  
CREATE TABLE plants (  
    plant_id SERIAL PRIMARY KEY,  
    ...  
);  
  
-- User_Plants table creation  
CREATE TABLE user_plants (  
    user_plant_id SERIAL PRIMARY KEY,  
    ...  
);  
  
-- Plant_Logs table creation  
CREATE TABLE plant_logs (  
    plant_log_id SERIAL PRIMARY KEY,  
    ...  
);
```

```
INSERT INTO users (name, email, password, gender,  
birth_date) VALUES  
(  
'John', 'john123@gmail.com',  
'$2a$10$vYR4pPQqR/oZcUDZfXrahecEejQHY0kLkDB5s.FctPRMcEM  
h1PYhG', 'M', '1988-05-01')  
...  
  
INSERT INTO plants (plant_name, plant_type, plant_desc,  
image_url, temperature_low, temperature_high,  
humidity_low, humidity_high, watering_interval)  
VALUES  
(  
'아이비', '덩굴식물', '아이비는 ...'  
...  
  
INSERT INTO user_plants (user_id, plant_id,  
plant_nickname) VALUES  
(1, 1, '노을이'), (1, 2, '햇님'), (2, 1, '별빛')  
...  
  
INSERT INTO plant_logs (user_plant_id, log_date, note,  
watered) VALUES  
(1, '2023-03-22', '관리가 어려워서 죽었습니다',  
false)...
```

1

2

LEAFY

식물 관리 웹 애플리케이션

Postgre
SQL

LEAFY-POSTGRESQL

config

postgresql.conf

init

init.sql

Dockerfile

LICENSE

script.sh

식물 정보

plants	
plant_name	varchar(50)
plant_type	varchar(50)
plant_desc	varchar(255)
image_url	varchar(255)
temperature_low	double precision
temperature_high	double precision
humidity_low	double precision
humidity_high	double precision
watering_interval	integer
created_at	timestamp
updated_at	timestamp
plant_id	integer

사용자 정보

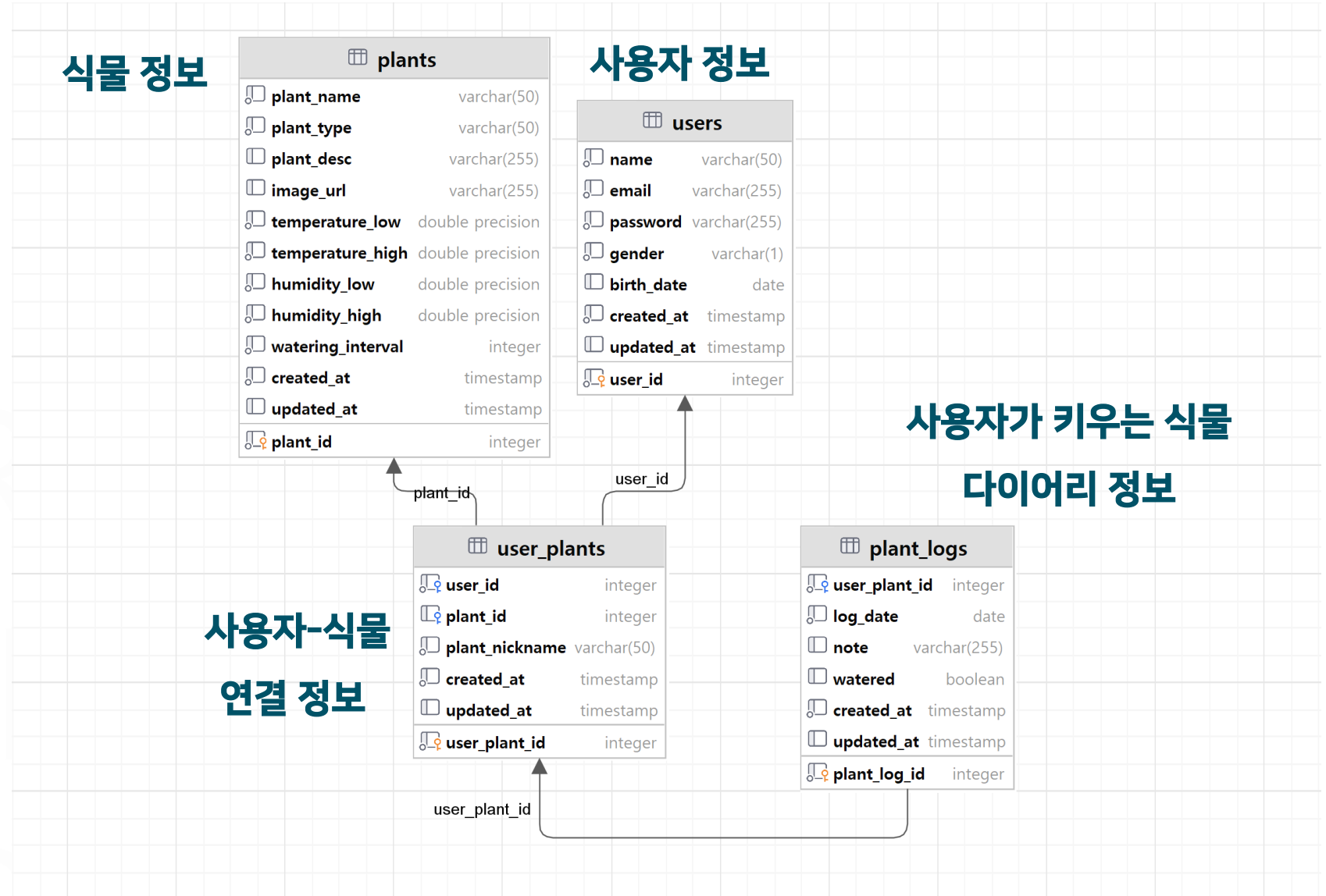
users	
name	varchar(50)
email	varchar(255)
password	varchar(255)
gender	varchar(1)
birth_date	date
created_at	timestamp
updated_at	timestamp
user_id	integer

사용자가 키우는 식물 다이어리 정보

plant_logs	
user_plant_id	integer
log_date	date
note	varchar(255)
watered	boolean
created_at	timestamp
updated_at	timestamp
plant_log_id	integer

사용자-식물 연결 정보

user_plants	
user_id	integer
plant_id	integer
plant_nickname	varchar(50)
created_at	timestamp
updated_at	timestamp
user_plant_id	integer



LEAFY

식물 관리 웹 애플리케이션

Postgre
SQL

LEAFY-POSTGRESQL

config

postgresql.conf

init

init.sql

Dockerfile

LICENSE

script.sh

leafy/leafy-postgresql/Dockerfile

#PostgreSQL 13 버전을 베이스 이미지로 사용

FROM postgres:13

#/docker-entrypoint-initdb.d/에 있는 sql문은 컨테이너가 처음 실행 시 자동실행됨

COPY ./init/init.sql /docker-entrypoint-initdb.d/

#기본 설정 파일을 덮어쓰기하여 새로운 설정 적용

COPY ./config/postgresql.conf /etc/postgresql/custom.conf

#계정정보 설정

ENV POSTGRES_USER=myuser

ENV POSTGRES_PASSWORD=mypassword

ENV POSTGRES_DB=mydb

EXPOSE 5432

CMD ["postgres", "-c", "config_file=/etc/postgresql/custom.conf"]

`docker cp` 원본위치 복사위치

컨테이너와 호스트 머신 간 파일 복사

`docker cp` 컨테이너명:원본위치 복사위치

컨테이너->호스트머신으로 파일 복사

`docker cp` 원본위치 컨테이너명:복사위치

호스트머신->컨테이너로 파일 복사

postgres컨테이너 실행(Shell1)

1. postgres:13 이미지 실행 및 셸 접근

```
docker run -d --name postgres -e POSTGRES_PASSWORD=password postgres:13
docker exec -it postgres bin/bash
```

2. 내부 파일시스템 확인

```
ls /
```

3. 초기 설정 파일 확인

```
cat /var/lib/postgresql/data/postgresql.conf
```

4. 초기 실행할 sql을 보관하는 폴더 확인

```
ls -al /docker-entrypoint-initdb.d
```

Shell2로 이동 >>>>

6. 복사한 sql문 실행

```
psql -U postgres -c "\d"
```

```
psql -U postgres -f /docker-entrypoint-initdb.d/init.sql
```

7. 데이터 확인

```
psql -U postgres -c "\d"
```

Shell2로 이동 >>>>

postgres 상태 확인 및 파일 복사(Shell2)

*현재 경로가 easydocker/leafy-postgresql 경로인지 확인

5. 컨테이너로 설정 파일과 sql문 복사

```
docker cp ./init/init.sql postgres:docker-entrypoint-initdb.d/
```

```
docker cp ./config/postgresql.conf postgres:etc/postgresql/postgresql.conf
```

<=== Shell1로 이동

8 컨테이너 삭제

```
docker rm -f postgres
```

>>>> 실습 종료

Leafy PostgreSQL 빌드 및 실행, 테스트

1. network 가 제대로 생성되어 있는지 확인

```
docker network ls
```

```
docker network create leafy-network
```

2. 이미지 빌드 및 푸시 (easydocker/leafy/leafy-postgresql 경로에서 실행)

```
docker build -t (개인레지스트리명)/leafy-postgres:1.0.0 . --no-cache
```

```
docker push (개인레지스트리명)/leafy-postgres:1.0.0
```

3. 컨테이너 실행 및 로그 확인

```
docker run -d --name leafy-postgres --network leafy-network (개인레지스트리명)/leafy-postgres:1.0.0
```

```
docker logs leafy-postgres
```

4. 컨테이너 접근 및 DB 명령어 실행

```
docker exec -it leafy-postgres su postgres bash -c "psql --username=myuser --dbname=mydb"
```

5. 초기 데이터 조회 (Ctrl+C 로 종료)

```
mydb=# SELECT * FROM users;
```

```
mydb=# SELECT * FROM plants;
```

```
mydb=# SELECT * FROM user_plants;
```

```
exit
```

LEAFY

식물 관리 웹 애플리케이션

PostgreSQL

LEAFY-POSTGRESQL

config

postgresql.conf

init

init.sql

Dockerfile

LICENSE

script.sh

Layers (31)

17	ENV PGDATA=/var/lib/postgresql/data	0 B	✓
18	mkdir -p "\$PGDATA" && chown -R postgres:postgres "\$PGDATA" && chmod 777 ...	0 B	✓
19	VOLUME [/var/lib/postgresql/data]	0 B	✓
20	COPY file:512acb0aab31f9e5d908f16e2f4478f65cddd5d4e555a02a1551074b...	12.48 KB	✓
21	ENTRYPOINT ["docker-entrypoint.sh"]	0 B	✓
22	STOPSIGNAL SIGINT	0 B	✓
23	EXPOSE 5432	0 B	✓
24	CMD ["postgres"]	0 B	✓
25	COPY ./init/init.sql /docker-entrypoint-initdb.d/ # buildkit	17.29 KB	✓
26	COPY ./config/postgresql.conf /etc/postgresql/postgresql.conf # buildkit	1.39 KB	✓
27	ENV POSTGRES_USER=myuser	0 B	✓
28	ENV POSTGRES_PASSWORD=mypassword	0 B	✓
29	ENV POSTGRES_DB=mydb	0 B	✓
30	EXPOSE map[5432/tcp:{}]	0 B	✓

기본

PostgreSQL

이미지

+(커스텀레이어)

설정파일,

초기sql 및

DB 접속정보 설정

PART5. 컨테이너 애플리케이션 구성

Leafy 백엔드 컨테이너

LEAFY

식물 관리 웹 애플리케이션

Leafy



leafy.jar

백엔드 Spring Boot Application 구성

5. 애플리케이션 실행

```
java -jar leafy.jar
```

4. 의존성 라이브러리 설치 및 빌드

```
gradle clean build
```

3. 소스코드 다운로드

```
git clone ...
```

2. 빌드 도구 설치

```
gradle
```

1. OS구성 및 Java Runtime 설치

```
Java Runtime
```

```
OS
```


LEAFY

식물 관리 웹 애플리케이션

Leafy

CMD ["java", "-jar",
"leafy.jar"]

FROM openjdk:11-jre-slim

실행 스테이지

백엔드 Spring Boot Application 구성

5. 애플리케이션 실행

java -jar leafy.jar



4. 의존성 라이브러리 설치 및 빌드

gradle clean build

3. 소스코드 다운로드

git clone ...

2. 빌드 도구 설치

gradle

1. OS구성 및 Java Runtime 설치

Java Runtime

OS

RUN gradle clean
build

COPY src .

FROM gradle:6.9.1
AS build

빌드 스테이지

LEAFY

식물 관리 웹 애플리케이션

Leafy

소스코드 다운로드

1. leafy/leafy-backend 디렉토리로 이동, 파일 구조 확인

```
cd ../leafy-backend
```

소스코드 구조

leafy C:\Users\dewiki\Desktop\leafy\leafy

- java
 - com.dewiki.leafy
 - config
 - controller ----- 특정 경로 요청에 대한 응답 전달
 - dto ----- 데이터 전달에 필요한 객체 정의
 - exception
 - model ----- DB 테이블구조에 맞는 객체 정의
 - repository ----- JPA를 활용한 Repository 정의
 - service ----- 실제 응답 내용을 작성하기 위한 비즈니스 로직
 - LeafyApplication
 - resources
- test
- .gitignore
- build.gradle ----- 애플리케이션의 의존 라이브러리 정보
- Dockerfile ----- 애플리케이션 이미지를 빌드하기 위한 도커파일
- gradlew

LEAFY

식물 관리 웹 애플리케이션

Leafy

소스코드 구조

```
@RestController
@RequestMapping("/api/v1/plants")
@RequiredArgsConstructor
public class PlantController {

    @GetMapping("") /api/v1/plants GET 요청시 전체 식물 리스트 응답
    public ResponseEntity<List<PlantDetailDto>> getAllPlants() {
        List<PlantDetailDto> plantDetailDtoList = plantService.getAllPlants();
        return new ResponseEntity<>(plantDetailDtoList, HttpStatus.OK);
    }
}

.....

@Service
@RequiredArgsConstructor
public class PlantService {

    public List<PlantDetailDto> getAllPlants() {
        return plantRepository.findAll()
            .stream()
            .map(PlantMapper::toDetailDto)
            .sorted(Comparator.comparing(PlantDetailDto::getCreatedAt).reversed())
            .collect(Collectors.toList());
    }
}
```

Database에서 전체 식물 리스트를 불러와
생성일자 순으로 정렬

LEAFY

식물 관리 웹 애플리케이션



도메인	API명	METHOD	URI
식물	모든 식물 조회	GET	/api/v1/plants
	식물 ID로 조회	GET	/api/v1/plants/{plantId}
	식물 추가	POST	/api/v1/plants
	식물 수정	PUT	/api/v1/plants/{plantId}
	식물 삭제	DELETE	/api/v1/plants/{plantId}
사용자	모든 사용자 조회	GET	/api/v1/users
	사용자 ID로 조회	GET	/api/v1/users/{userId}
	사용자 추가	POST	/api/v1/users
	사용자 수정	PUT	/api/v1/users/{userId}
	사용자 삭제	DELETE	/api/v1/users/{userId}
	사용자 로그인 처리	POST	/api/v1/users/login

LEAFY

식물 관리 웹 애플리케이션



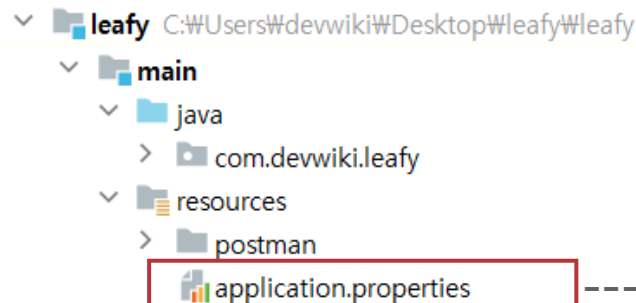
도메인	API명	METHOD	URI
사용자-식물	특정 사용자의 모든 식물 조회	GET	/api/v1/user-plants/user/{userId}
	특정 사용자가 소유한 특정 식물 조회	GET	/api/v1/user-plants/{userPlantId}
	특정 사용자가 소유한 식물 추가	POST	/api/v1/user-plants
	특정 사용자의 식물 소유 정보 수정	PUT	/api/v1/user-plants/{userPlantId}
	특정 사용자의 식물 소유 정보 삭제	DELETE	/api/v1/user-plants/{userPlantId}
식물일기	모든 식물 일기 조회	GET	/api/v1/plant-logs
	특정 사용자의 모든 일기 조회	GET	/api/v1/plant-logs/user/{userId}
	특정 사용자의 최근 5건 일기 조회	GET	/api/v1/plant-logs/recent/user/{userId}
	식물 일기 ID로 조회	GET	/api/v1/plant-logs/{plantLogId}
	식물 일기 추가	POST	/api/v1/plant-logs
	식물 일기 수정	PUT	/api/v1/plant-logs/{plantLogId}
	식물 일기 삭제	DELETE	/api/v1/plant-logs/{plantLogId}

LEAFY

식물 관리 웹 애플리케이션

Leafy

환경변수 설정 (leafy-backend\src\main\resources\application.properties)



----- 애플리케이션의 구성 정보 관리

PostgreSQL Database

spring.datasource.driver-class-name=org.postgresql.Driver

spring.datasource.url=jdbc:postgresql://\${DB_URL:localhost}:\${DB_PORT:5432}/\${DB_NAME:mydb}

spring.datasource.username=\${DB_USERNAME:myuser}

spring.datasource.password=\${DB_PASSWORD:mypassword}

Hibernate

spring.jpa.show-sql=true

spring.jpa.hibernate.ddl-auto=none

\${시스템환경변수명:기본값}

\${DB_URL:localhost} : 데이터베이스의 URL

\${DB_PORT:5432} : 데이터베이스의 PORT

\${DB_NAME:mydb} : 데이터베이스명

\${DB_USERNAME:myuser} : 사용자계정

\${DB_PASSWORD:mypassword} : 접속 비밀번호

gradle 컨테이너 실행(Shell1)

(leafy/leafy-backend 경로에서 실행)

1. gradle:7.6.1-jdk17 이미지 실행 및 셸 접근

```
docker run -it --name gradle gradle:7.6.1-jdk17 bash
```

2. 작업 경로 생성 및 이동

```
mkdir /app && cd /app
```

Shell2로 이동 ==>

4. 복사된 소스코드 확인

```
ls /app
```

```
ls build/libs
```

5. 애플리케이션 빌드

```
gradle clean build -no-daemon
```

6. 생성된 jar 아티팩트 파일 확인

```
ls build/libs
```

7. 애플리케이션 실행

```
java -jar build/libs/Leafy-0.0.1-SNAPSHOT.jar
```

Shell2로 이동 ==>

gradle 상태 확인 및 파일 복사(Shell2)

(leafy/leafy-backend 경로에서 실행)

3. 컨테이너로 소스코드 복사

```
docker cp . gradle:app
```

<==== Shell1로 이동

8 컨테이너 삭제

```
docker rm -f gradle
```

====> 실습 종료

LEAFY

식물 관리 웹 애플리케이션



Leafy

easydocker/leafy/leafy-backend/Dockerfile

```
# 빌드 이미지로 OpenJDK 11 & Gradle을 지정
FROM gradle:7.6.1-jdk17 AS build

# 소스코드를 복사할 작업 디렉토리를 생성
WORKDIR /app

# 호스트 머신의 소스코드를 작업 디렉토리로 복사
COPY . /app

# Gradle 빌드를 실행하여 JAR 파일 생성
RUN gradle clean build --no-daemon

# 런타임 이미지로 OpenJDK 11 JRE-slim 지정
FROM openjdk:11-jre-slim

# 애플리케이션을 실행할 작업 디렉토리를 생성
WORKDIR /app

# 빌드 이미지에서 생성된 JAR 파일을 런타임 이미지로 복사
COPY --from=build /app/build/libs/*.jar /app/leafy.jar

EXPOSE 8080
ENTRYPOINT ["java"]
CMD ["-jar", "leafy.jar"]
```


Leafy 빌드 및 실행, 테스트

1. 이미지 빌드 및 푸시 (easystack/leafy/leafy-backend 경로에서 실행)

```
docker build -t (개인레지스트리명)/leafy-backend:1.0.0 .
```

```
docker push (개인레지스트리명)/leafy-backend:1.0.0
```

2. 컨테이너 실행 및 로그 확인

```
docker run -d -p 8080:8080 -e DB_URL=leafy-postgres --name leafy --network leafy-network (개인레지스트리명)/leafy-backend:1.0.0
```

```
docker logs leafy-backend
```

3. API 테스트

```
curl http://localhost:8080/api/v1/users
```

```
curl http://localhost:8080/api/v1/plant-logs/1
```

java -jar leafy.jar

DB_URL=leafy-postgres

Java Runtime

OS

application.properties

PostgreSQL Database

spring.datasource.driver-class-name=org.postgresql.Driver

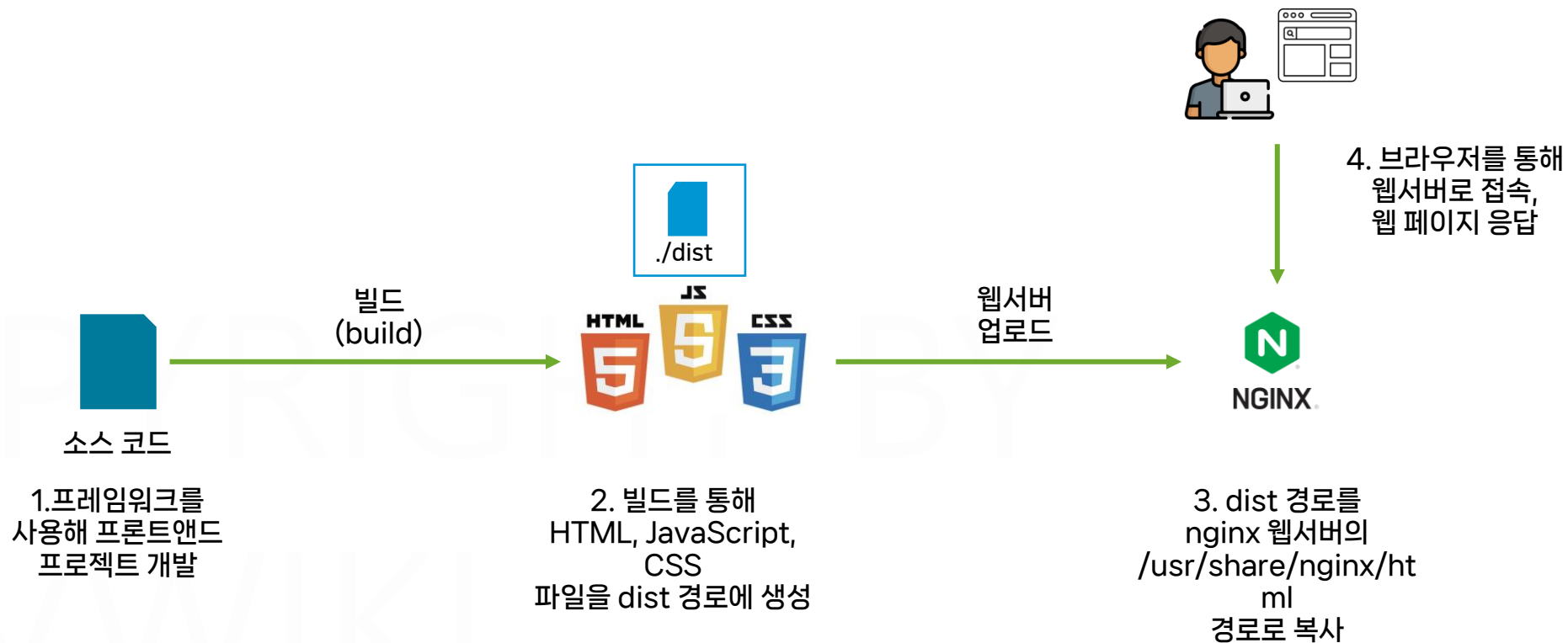
spring.datasource.url=jdbc:postgresql://leafy-postgres:5432/leafy-postgres

spring.datasource.username=\${DB_USERNAME:myuser}

spring.datasource.password=\${DB_PASSWORD:mypassword}

PART5. 컨테이너 애플리케이션 구성

Leafy 프론트엔드 컨테이너



LEAFY

식물 관리 웹 애플리케이션

Leafy
-front

프론트엔드 Vue.js WebServer 구성

6. 웹서버 실행

```
nginx -g daemon off;
```

5. 빌드 결과 폴더를

/usr/share/nginx/html 폴더로 복사

```
cp ./dist /usr/share/nginx/html
```

4. 의존성 라이브러리 설치 및 빌드

```
npm ci & npm run build
```

3. 소스코드 다운로드

```
git clone ...
```

2. 빌드 도구 설치

```
node.js, npm
```

1. OS구성 및 Nginx 웹 서버 설치

Nginx

OS



./dist

LEAFY

식물 관리 웹 애플리케이션

Leafy
-front

```
CMD ["nginx", "-g",  
"daemon off;"]
```

```
FROM nginx:1.21.4-alpine
```

실행 스테이지

프론트엔드 Vue.js
WebServer 구성

6. 웹서버 실행

```
nginx -g daemon off;
```

5. 빌드 결과 폴더를

/usr/share/nginx/html 폴더로 복사

```
cp ./dist /usr/share/nginx/html
```



./dist

4. 의존성 라이브러리 설치 및 빌드

```
npm ci & npm run build
```

3. 소스코드 다운로드

```
git clone ...
```

2. 빌드 도구 설치

```
node.js
```

1. OS구성 및 Nginx 웹 서버 설치

Nginx

OS

```
RUN npm run build
```

```
RUN npm ci
```

```
COPY . .
```

```
FROM node:14
```

```
AS build
```

빌드 스테이지

LEAFY

식물 관리 웹 애플리케이션

Leafy
-front

소스코드 다운로드

1. leafy/leafy-frontend 디렉토리로 이동
`cd ../leafy-frontend`

소스코드 구조

▼ LEAFY-FRONT	
> public	
▼ src	
> api	
> components	----- 사용자에게 응답할 페이지(재사용)
▼ router	
JS router.js	----- 특정 경로 요청에 대한 파일 정의
> store	
> views	----- 사용자에게 응답할 페이지
▼ App.vue	
JS main.js	
.dockerignore	
.gitignore	
babel.config.js	
Dockerfile	----- 애플리케이션 이미지를 빌드하기 위한 도커파일
jsconfig.json	
LICENSE	
package-lock.json	
package.json	----- 애플리케이션의 의존 라이브러리 정보

LEAFY

식물 관리 웹 애플리케이션

Leafy
-front

요청 URL과 응답할 파일 지정 (leafy-frontend\src\router\router.js)

```
const routes = [
  {
    path: '/',
    name: 'HomePage',
    component: () => import('@views/HomePage.vue'),
  },
  {
    path: '/plants',
    name: 'PlantList',
    component: () => import('@views/PlantList.vue'),
  },
  {
    path: '/plants/add',
    name: 'PlantAdd',
    component: () =>
      import('@components/modals/PlantAddModal.vue'),
  },
  {
    path: '/plantlogs',
    name: 'PlantlogList',
    component: () =>
      import('@views/PlantlogList.vue'),
  },
  {
    path: '/setting',
    name: 'UserCard',
    component: () => import('@views/UserCard.vue'),
  },
  {
    path: '/edituser',
    name: 'UserEdit',
    component: () => import('@views/UserEdit.vue'),
  },
  {
    path: '/login',
    name: 'UserLogin',
    component: () => import('@views/UserLogin.vue'),
  },
]
```

처리할 경로를 정의

전달할 파일을 정의

LEAFY

식물 관리 웹 애플리케이션

Leafy
-front

/로 요청 시 응답할 메인 페이지 (leafy-frontend/src/views/HomePage.vue)

`const fetchRecentLogs = async () => {` 최근 5건의 사용자 식물 일기 조회

`const userId = user.value.userId;`

`api.get(`/api/v1/plant-logs/recent/user/${userId}`)` 백엔드 API를 통해 데이터를 받아올 URI 정의

`.then(response => {`

`state.logs = response.data;` 백엔드 API로 부터 받은 응답을 저장

`})`

`.catch(error => {`

`console.error(error);`

`});`

`};`

백엔드 API로부터 받은 데이터를
HTML 페이지로 사용자에게 전달

`<h3>최근 일기</h3>`

`<div class="log-container">`

`<v-card v-for="log in logs" :key="log.plantLogId" class="log-card" variant="outlined">`

`...`

`</v-card>`

`</div>`

최근 다이어리

노을이(아이비)

1주일에 한 번 비료를 주기로 했
습니다

2023-03-24

햇님(스투키)

물을 적게 주도록 조절했습니다

2023-03-24



노을이(아이비)

새로운 아이비를 구입했습니다

2023-03-23



햇님(스투키)

더 밝은 곳으로 옮겼습니다

2023-03-23

LEAFY

식물 관리 웹 애플리케이션



페이지 명	URI	페이지 설명
로그인 페이지	/login	
메인 페이지	/	나의 식물 리스트와 최근 다이어리 5건 조회
전체 식물 조회	/plants	전체 식물 리스트 조회
식물 추가	/plants/add	새로운 식물 추가
나의 일기 조회	/plantlogs	로그인한 사용자의 일기 조회
설정 변경	/setting	
사용자 정보 변경	/edituser	로그인한 사용자의 설정 변경

node 컨테이너 실행(Shell1)

(leafy/leafy-frontend 경로에서 실행)

1. npm 이미지 실행 및 쉘 접근

```
docker run -it --name node node:14 bin/bash
```

2. 작업 경로 생성 및 이동

```
mkdir /app && cd /app
```

Shell2로 이동 ==>

4. 복사된 소스코드 확인 및 소스 빌드

```
ls
```

```
npm ci
```

```
npm run build
```

5. 생성된 아티팩트 파일 확인

```
ls dist
```

Shell2로 이동 ==>

8. nginx 이미지 실행 및 쉘 접근

```
docker run -d -p 80:80 --name nginx nginx
```

Shell2로 이동 ==>

node 컨테이너에서 빌드한 dist 파일 복사(Shell2)

(leafy/leafy-frontend 경로에서 실행)

3. 컨테이너로 소스코드 복사

```
docker cp . node:app
```

<==== Shell1로 이동

6. 빌드 파일 호스트로 복사

```
docker cp node:app/dist .
```

7. node 컨테이너 삭제

```
docker rm -f node
```

<==== Shell1로 이동

9. 빌드 파일 nginx로 복사

```
docker cp ./dist/. nginx:usr/share/nginx/html
```

10. 로컬호스트 접속 테스트 및 컨테이너 삭제

```
docker rm -f nginx
```

====> 실습 종료

LEAFY

식물 관리 웹 애플리케이션

Leafy
-front

easydocker/leafy/leafy-frontend/Dockerfile

```
# 빌드 이미지로 node:14 지정
FROM node:14 AS build

WORKDIR /app

# 빌드 컨텍스트의 소스코드를 작업 디렉토리로 복사, 라이브러리 설치 및 빌드
COPY . /app
RUN npm ci
RUN npm run build

# 런타임 이미지로 nginx 1.21.4 지정, /usr/share/nginx/html 폴더에 권한 추가
FROM nginx:1.21.4-alpine

# 빌드 이미지에서 생성된 dist 폴더를 nginx 이미지로 복사
COPY --from=build /app/dist /usr/share/nginx/html

EXPOSE 80
ENTRYPOINT ["nginx"]
CMD ["-g", "daemon off;"]
```

Leafy-front 빌드 및 실행, 테스트

1. 이미지 빌드 및 푸시 (easydocker/part5/leafy 경로에서 실행)

```
docker build -t (개인레지스트리명)/leafy-frontend:1.0.0 .
```

```
docker push (개인레지스트리명)/leafy-frontend:1.0.0
```

2. 컨테이너 실행 및 로그 확인, localhost 접속 테스트

```
docker run -d -p 80:80 --name leafy-front --network leafy-network (개인레지스트리명)/leafy-frontend:1.0.0
```

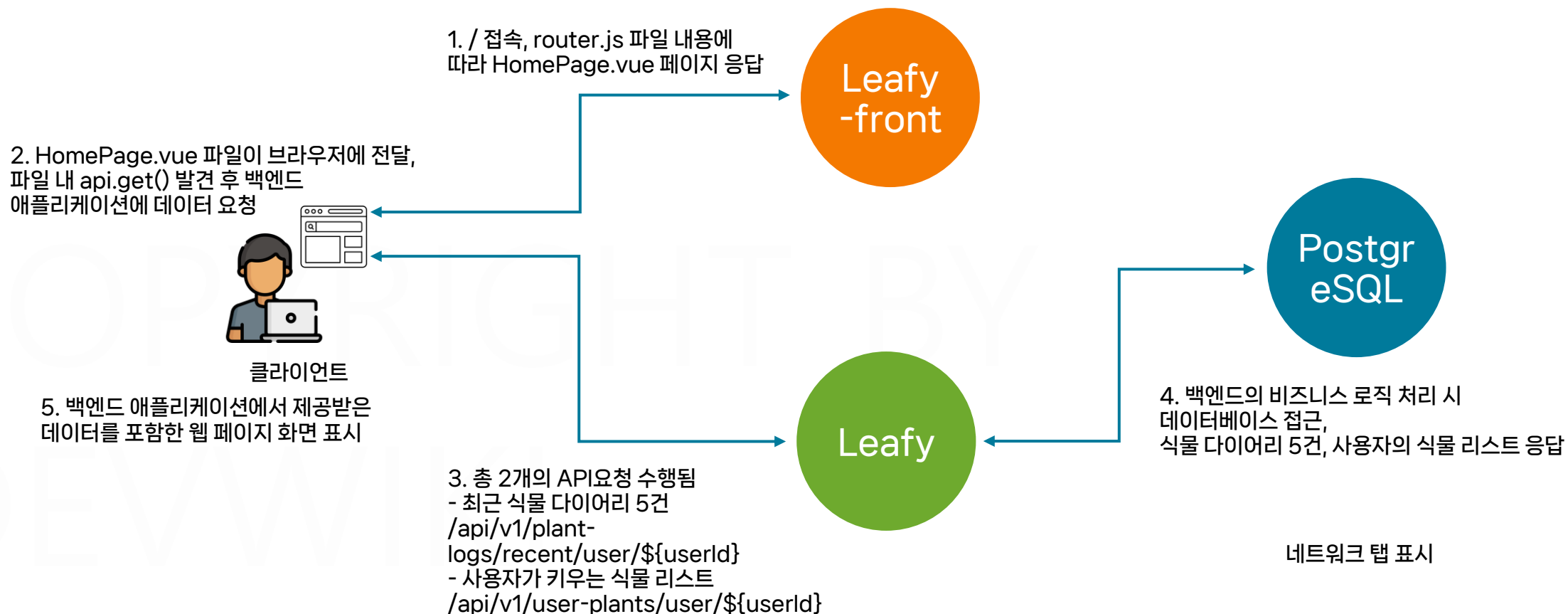
```
docker logs -f leafy-front
```

PART5. 컨테이너 애플리케이션 구성

Leafy 애플리케이션 테스트

LEAFY

식물 관리 웹 애플리케이션



Leafy 환경 정리

1. 정상 실행 확인 후 구성 환경 삭제

```
docker rm -f leafy-front leafy leafy-postgres
```

COPYRIGHT BY
DEVWIKI
COPYRIGHT BY